



白皮书

# Codex 长线工作实践

Codex 如何让工作在单次提示之外  
持续推进

**OpenAI**

# 目录

---

01 02 03

[持久线程]

[语音输入]

[引导]

---

04 05 06

[记忆]

[计算机与  
浏览器操作]

[远程控制]

---

07 08 09

[线程  
自动化]

[三个循环  
示例]

[目标]

---

10

[侧边栏]

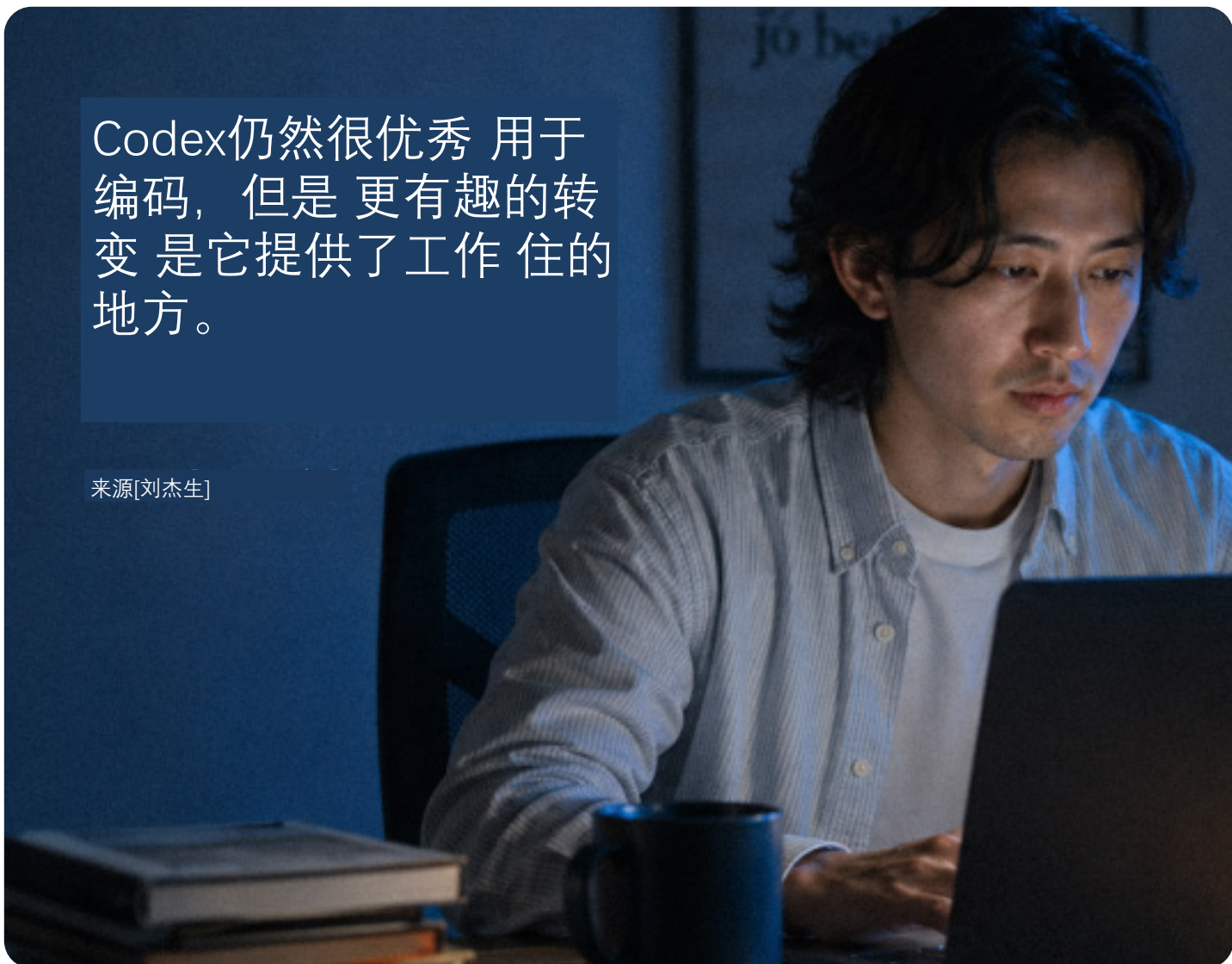
# Codex 是为 [编码工作] ：制作 差异，更改代码 仓库仓库， 审查变更， 并帮助发布代码。

但更广泛的模式是 开始看起来不同 每个人——不仅仅是开发人员。

当 Codex 拥有耐用的线程时， 共享内存、工具访问、一种重复的方式和一个地方 检查输出本身、工作 可以继续跨越更多 比一个提示。

Codex仍然很优秀 用于  
编码，但是 更有趣的转  
变 是它提供了工作 住的  
地方。

来源[刘杰生]



创建者 [Jason Liu] 使用 Codex 作为他的一部分 日常工作流  
程，来自演示文稿和文字记录 电子表格、浏览器任务和重  
复工作。

该指南介绍了 Jason 如何使用 Codex作  
为收集背景的地方，寻求帮助，回顾  
接下来发生的事情 返回，并返回下一  
步 不丢失线程。

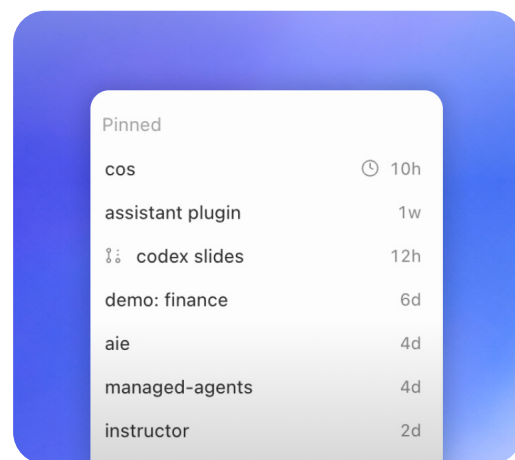
# 持久线程

01

## 给工作一个安放之处

对重要工作流来说，持久线程可以成为工作的归处：背景、偏好、过往决策，以及未闭环事项都会在此沉淀。

把 [持久线程] 用在  
你会反复回看的工作上：



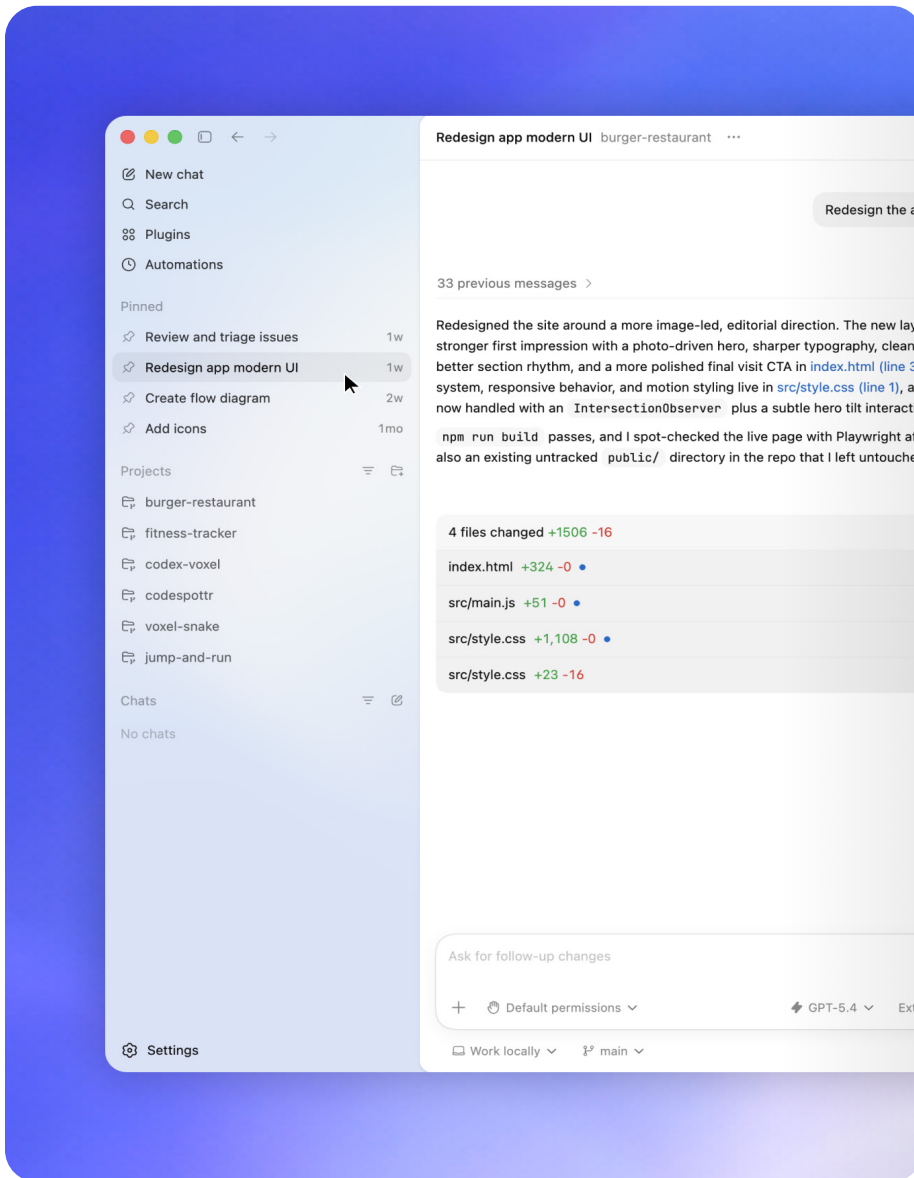
[幕僚参谋]  
消息、跟进、  
会议，以及  
未闭环事项。

[OpenAI CLI]  
命令、约定、  
仓库结构，以及  
查看偏好。

[社交反馈  
监控]  
反复问题、  
有用反馈，以及  
来自公开讨论的  
外部信号。

[Agents SDK]  
示例、文档、  
反复问题、  
与产品行为。

[面向 Codex 的  
开源项目]  
issue、维护者、  
贡献模式，  
以及发行说明。



持久线程使历史记录可用 随着对话的深入和存储 他们在记忆库中。但有 权衡：长时间运行的线程携带 上下文，运行成本可能比 新鲜的短线。对于重要的 工作流程，连续性是值得的 并且更容易管理。

# 语音输入

第 02 节

## 多放点自己的实际情况 思考Codex

[语音输入] 改变 Codex 获取的上下文类型。

好处不仅仅是速度。就是这么说的 输入通常包括作品的未编辑版本：记不清的名字，模糊的方向，不确定性，会很尴尬的事情 打字却很自然地说。



[语音输入] “我认为有一个叫本的人在 Slack 中提到这一点，我不记得了到底是什么，就去看看吧。”

Click to dictate or hold

^↑D





这是一个有用的指令，因为它是真正的工作通常是如何开始的。杰森将开始 当他浏览页面时 录制语音笔记 在浏览器中制作的代理。当他在完成后，他按 Enter 键，以便 Codex 可以 执行该反馈。

转录本的工作原理相同。一个电话，会议、走廊谈话或粗暴的谈话 语音笔记可以成为起始材料。Codex 可以帮助将原始版本转变为计划、草案、工件或下一步行动。

当很多计划变得更好时 该模型可以访问凌乱的 你的想法的版本。

来源[刘杰生]



# 引导

第 03 节

## 在 Codex 工作期间调整队列

配对后语音变得更有用 与[引导]。

引导意味着添加下一条指令 Codex 已经开始工作。你可以纠正方向，添加上下文、批准下一步或排队 工具调用后的下一步操作。当你复习的时候 一个网站，听起来可能像：

⌘=

[引导] “把这个改小一点。” “这个副本是错误的。” “这两件事之间的距离 感觉不太对劲。” “完成后，打开 PR。” “等待预览部署。” “给我看看之前的预览链接 什么都发布了。”

Q Explored 2 searches, ran 2 commands

Next I'm making the file edits: project metadata, README instructions. The UI will be compact and have more controls, and responsive layout.

Creating `tsconfig.json` +24 -0

Once this is done, open a PR.

+

🎤

↑

# 记忆

第 04 节

## 让记忆成为某种东西 你可以回顾一下

内存充当笔记本，为以下内容提供上下文 行动。随着线程持续时间更长，它们需要[内存] 谈话之外。当消息历史记录 有帮助，但这并不总是足够的。有用的上下文应该 成为您可以打开、编辑、比较和重用的东西。

```
• vault/  
  ├── AGENTS.md          # How the vault operates  
  ├── TODO.md            # Cross-project priorities and follow-ups  
  |  
  ├── projects/  
  |   ├── README.md      # Active-project index  
  |   ├── agents-sdk/  
  |   ├── codex-for-open-source/  
  |   ├── early-access-program/  
  |   ├── rust-migration-blog/  
  |   └── ...             # Other active and archived workstreams  
  |  
  ├── agent/  
  |   ├── USER_CONTEXT.md # Working preferences and context  
  |   ├── daily-summary-*.md # Daily decisions and follow-ups  
  |   └── ...             # Research, synthesis, and learning  
  |  
  ├── people/  
  |   ├── <username>.md   # Person and relationship context  
  |   └── ...  
  |  
  └── notes/
```



[记忆] 一种模式是一个金库：保险库/  
—— TODO.md |  
—— 人/ |—— 项目/  
—— 代理/  
—— 笔记/

记忆库可以容纳滚动 你的工作背景：  
人、决策、未闭环事项、 每日  
笔记、项目状态以及 否则的细节  
在线程之间迷失。

保持这种区别清晰： 代码仓  
库库库保存代码。 保险库保  
持滚动 围绕工作的背景。

当记忆库位于 GitHub 时，  
差异成为审查表面  
为了记忆。你可以看到什么  
Codex认为很重要 足以写下来。

该审查步骤很重要。 长时间运  
行的线程应该 不悄悄积累模糊  
谈话中的印象

历史。他们应该记录  
发生了什么变化：



[记录]  
这个人更喜欢这个。  
这个项目正在等待。 做出了这个决定。  
  
这个循环是闭合的。



【记忆说明】  
正如人们所提到的，  
更新相关内容 人们注意到。 随着项  
目的推进， 更新项目页面。 当循环  
关闭时， 将它们标记为关闭。 当决  
定做出时， 写下决定 以及为什么它  
很重要。

# 计算机与浏览器操作

第05节

## 决定什么 线可以接触

一旦一个线程拥有了内存，下一个问题很实际：

【它能用来做什么？】

连接器将 Codex 扩展到 经常工作的地方

首先出现：松弛线程，收件箱、日历、文档和问题跟踪器。

对于此工作流程，它有帮助分离表面：



[\$浏览器]

本地网络表面，预览，和注释



[@chrome] 已登录的浏览器

会议和经过身份验证的选项卡



[@计算机] 仅 GUI 工作

需要点击



[连接器] 松弛、Gmail、日历、GitHub 和其他工作表面



[技能] 可重复使用的工作流程 Codex可以重复

如果您正在本地应用程序上进行迭代，请使用 浏览器表面。如果任务取决于 登录状态或多个经过身份验证的选项卡，使用 Chrome。如果完成任务的唯一方法 通过桌面应用程序，使用计算机使用 明确权限和审核。

技能使重复工作更容易重用。一旦工作流程成功，您通常可以 包装说明、参考资料、和脚本，所以 Codex 不需要 每次都重教。

Look across my  Google Calendar ,  Slack , and  Google Drive and create a crisp leadership brief with what changed, what needs attention, and what happens next

+

GPT-5.5

High

▼

|          |                                                                                                                                                                                                                 |                     |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| Calendar | <ul style="list-style-type: none"><li>Event insights now show attendance likelihood to help with planning.</li><li>Improved booking experience with guest self-scheduling and time zone intelligence.</li></ul> | Helps more e reduce |
|          | <ul style="list-style-type: none"><li>AI-powered thread summaries are now available in channel view.</li></ul>                                                                                                  | Why i               |

# 远程控制

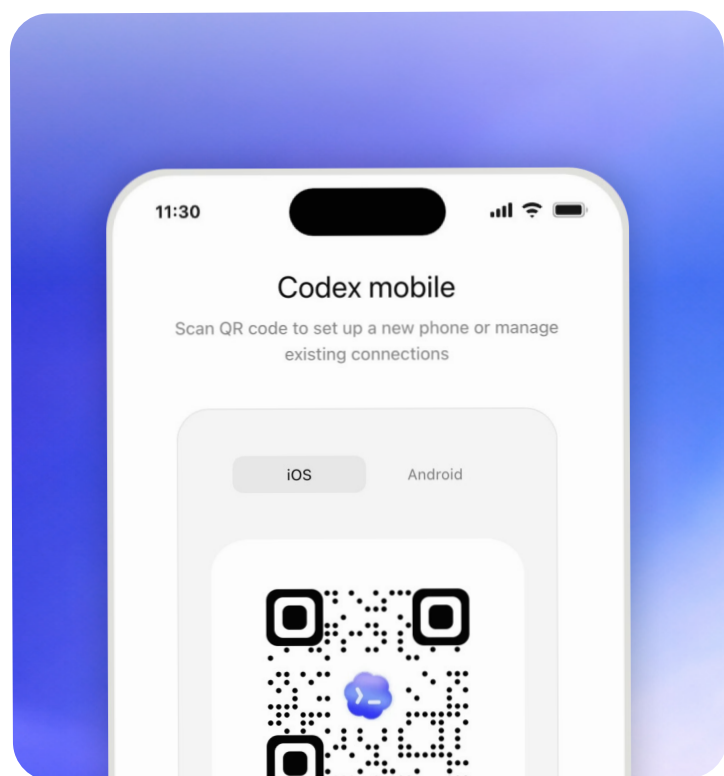
第 06 节

## 保持更长时间 循环便携式

【远程控制】搞定 更容易靠近 长时间运行的任务。

Codex 可以继续工作 您的文件所在的机器， 权限和本地设置 已经住了。您可以入住 从另一台设备， 查看 发现了什么， 回答问题， 批准下一步， 或改变方向。

当Codex委员会 已经在做某事 需要时间。



在办公桌上开始任务。走开。回顾下一个决定 从您的手机上点。批准， 重定向， 或要求不同的通行证。

来源[刘杰生]

远程控制不是理由 跳过评论。这是一种方法 保持足够的注意力 循环以解锁下一步动作。



# 线程自动化

第 07 节

## 附表Codex 返回

[线程自动化]是心跳- 附有样式重复叫醒服务 到当前线程。他们告诉食品Codex委员会 返回同一对话 节奏，而是保留上下文 每次都从头开始。

一个线程可以有多个调度。  
它可以运行直到满足条件。它可以 随着任务的变化调整节奏。

</>

【正常提示】  
现在就这样做。

相对

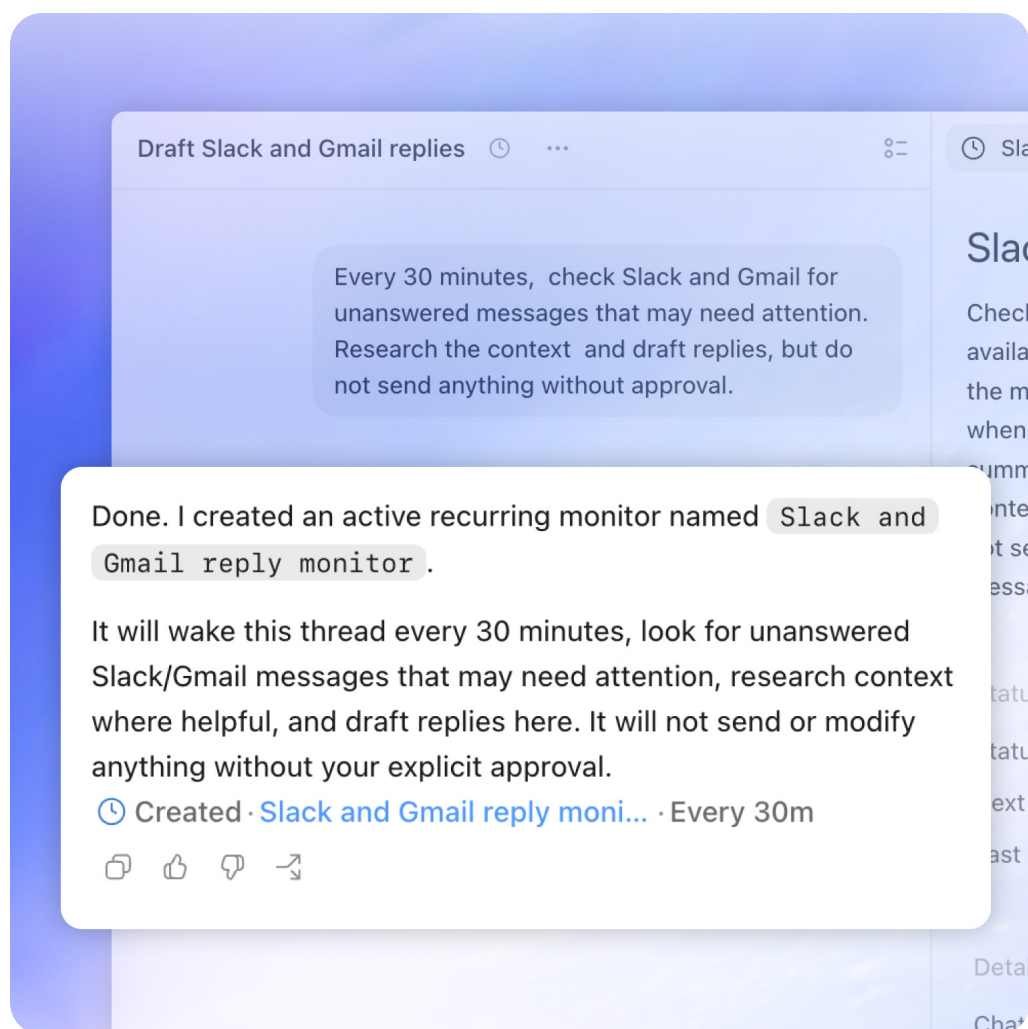
⋮

[线程自动化] 继续检查这个 并将其向前移动 有些事情发生了变化。

这可能意味着检查 拉取请求、Slack 线程、收件箱、部署、文档、客户支持线程，或 长时间运行的命令。



[线程自动化] 每30分钟，检查 Slack 和 Gmail 对于未回复的消息 这可能需要注意。研究背景 并草拟答复，但是 不要发送任何东西 未经批准。



# 三个循环示例

第 08 节

## 应用食典的真正方法 在您的 工作流程中

力量来自跨功能的[循环]：背景、工具、记忆、重现  
和回顾。



# 循环1 [参谋长]

Codex 按计划检查 Slack 和 Gmail，查找可能需要注意的消息，寻找它们背后的背景，并起草答复。用户仍然决定 发送的内容。

Working for 2m 23s

I'll set this up as a recurring monitor rather than trying to ha manually in this thread. I'm going to look for the automation then I'll check what Slack/Gmail access is available so the r actually inspect the right sources.

I found the scheduler. I'm also checking whether Slack and connectors are already callable in this workspace; that dete whether I can create a fully working monitor now or need to setup for you to approve/connect.

Awaiting approval

Every 30 minutes, check Slack and Gmail for unanswered messages that need my attention. Help me prioritize what matters most. If someone asks me a question, research the answer as deeply as you can and draft a reply for me, but do not send it.

+





循环1 [参谋长]

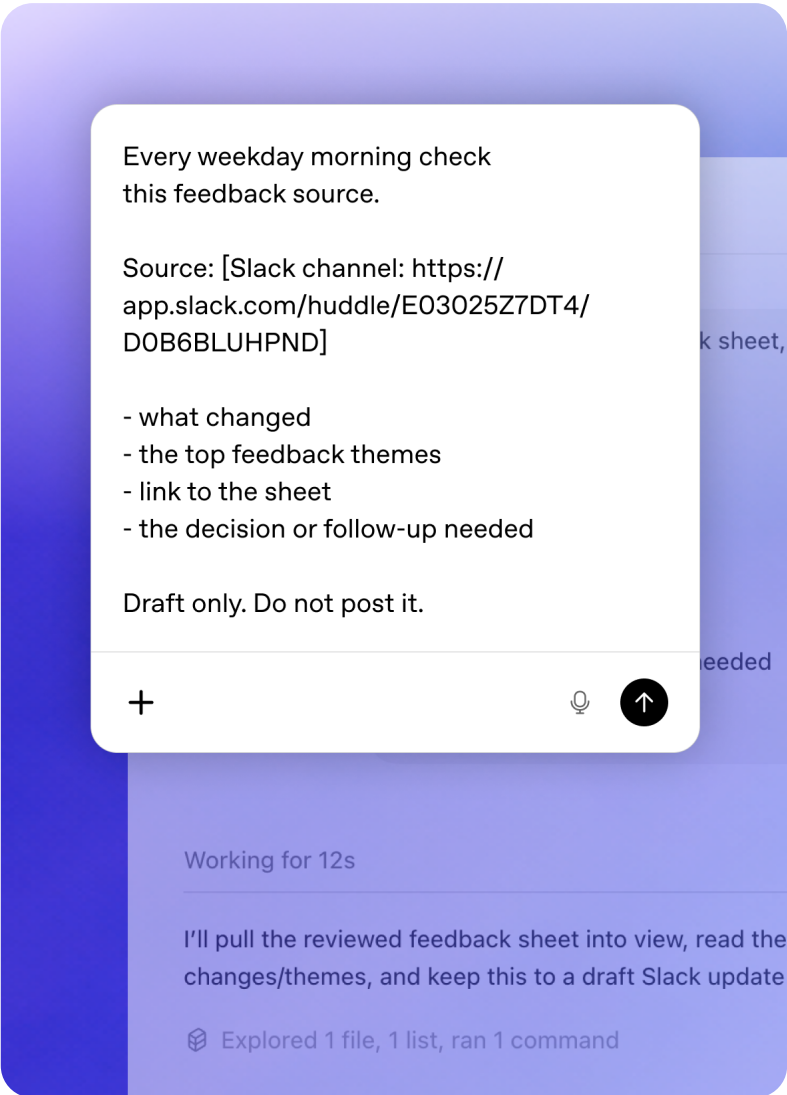
[Codex准备了什么] 打开消息  
相关背景 草稿答复 问题是 需要  
判断

[你决定] 批准 语气  
时机 最终决定

# 循环2 [监控反馈]

Codex 监视 Slack 线程的动画 反馈，更新 Remotion项目， 重新渲染该作品，并准备 修订 供审查。

循环跨越工具：用于反馈的 Slack， 远程渲染和计算机 当上传或审查表面时使用 需要 GUI。



## 循环2 【监控反馈】

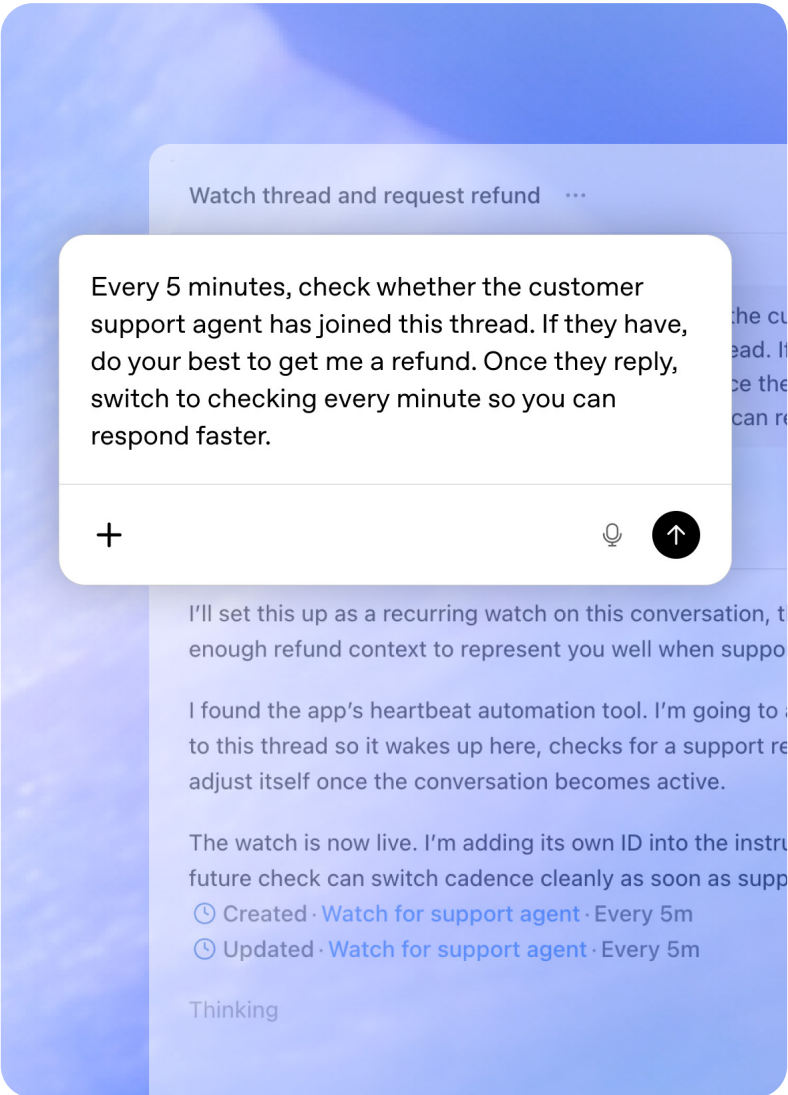
[Codex准备了什么] 反馈汇总  
更新渲染 修订说明 下一篇评论  
链接

[你决定] 创造性判断 最终批  
准 出版决定

# 循环3[获得退款]

Codex 检查客户是否支持 代理已加入，然后准备下一个 当对话发生变化时的响应。

当用户离开时任务可以继续， 但动作仍然受到限制。



循环3 [获得退款]

[Codex准备了什么] 状态检查  
答复草案 有用的证据 建议下一步

[你决定] 同意 批准 任何不可逆转  
的行动

# 目标

第 09 节

## 设定目标 Codex 可以验证

一个薄弱的目标要求食典委实施一项计划。

[更强大的目标] 为食典委提供了一些东西 测试依据：  
预期行为、审查标准、约束，或完成的明确定义。



[弱目标]

“实施  
计划  
在这个  
降价  
文件。”

相对



【坚定的目标】

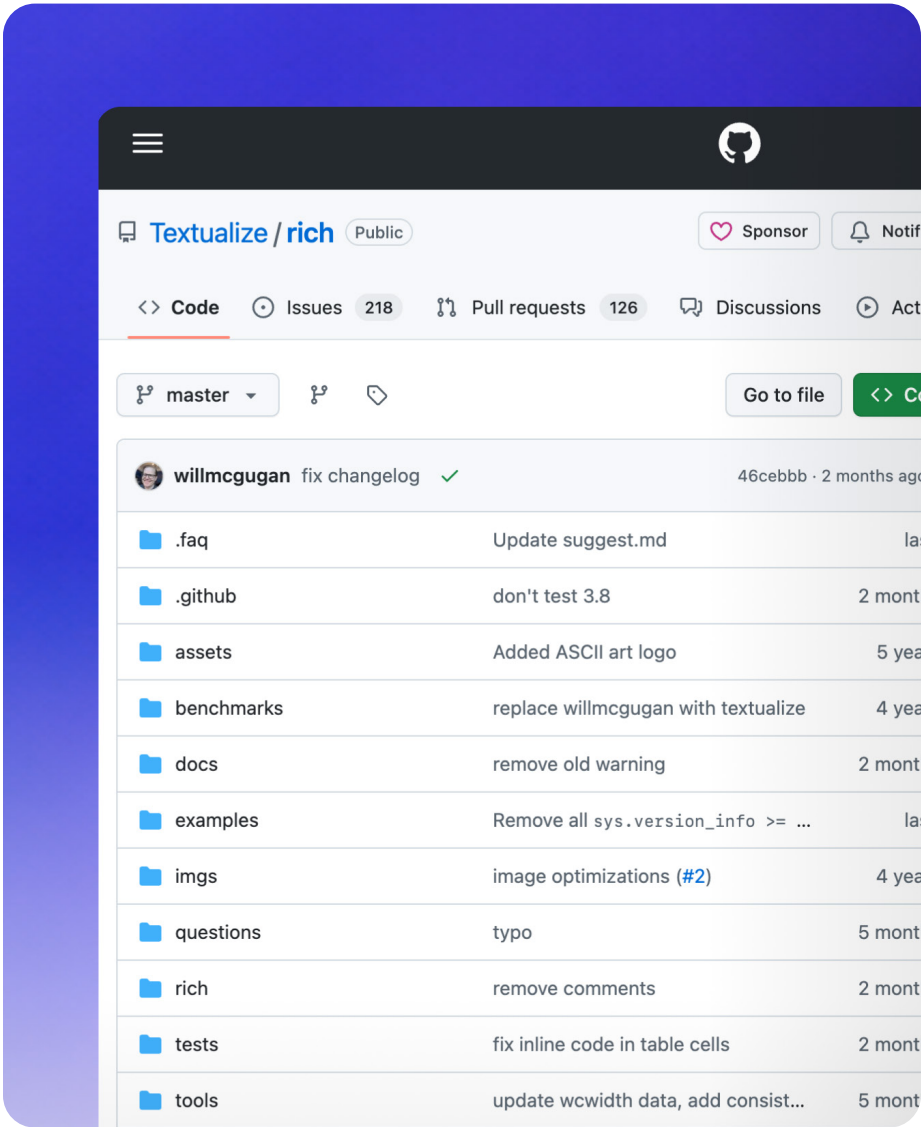
“移植这个图书馆，让公众  
API兼容，并使用原装  
单元测试作为成功检查。  
工作已准备好接受审查，如下：  
相同的测试通过并且  
差异已被记录下来。”



Rich-to-Rust 的例子使得 这个混凝土。目标不是 仅用于移植库。目标是以一种可以的方式移植它 通过原始单元测试。

该测试套件给了运行真正的 标准。工作还没有准备好 直到新的实施 通过了相同的测试。

迁移[丰富 进入铁锈]



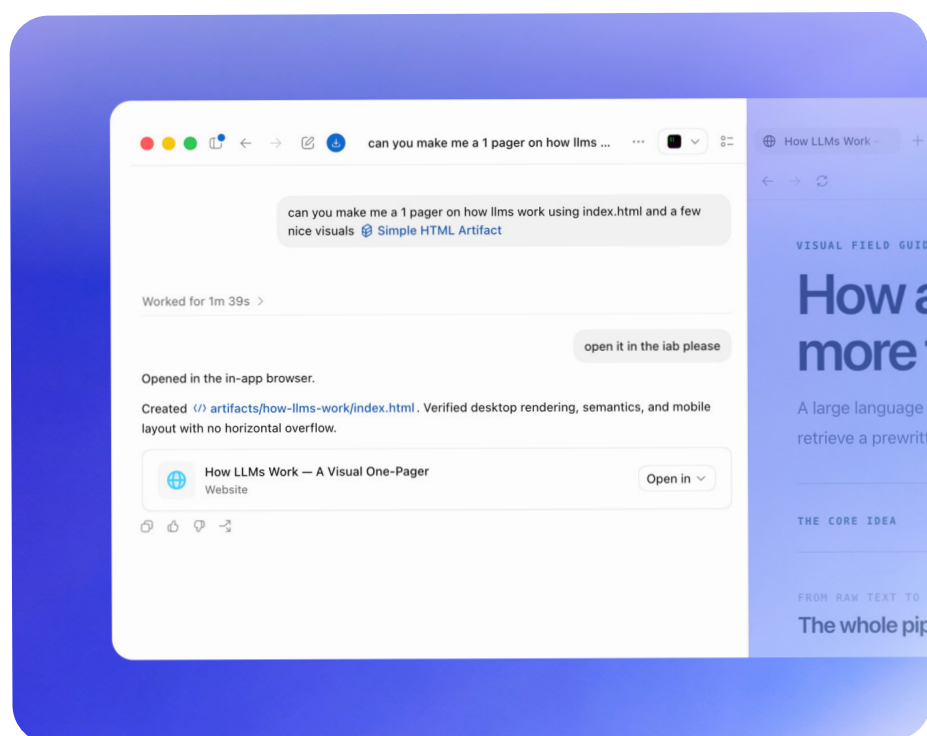
# 侧边栏

第 10 节

## 使工件成为循环的一部分

[侧面板]很容易理解为 预览表面，但这低估了它。这是哪里 Codex 不仅仅是一个聊天界面。您可以检查 Codex 正在运行的同一对象 打开、发表评论、查看更改并保留 线程内的工件。

[侧边栏]



[检查文物]  
降价，  
电子表格、CSV、  
PDF 和幻灯片可以  
都住在那里。  
降价可以  
审查与  
评论。电子表格  
可以渲染公式  
和支持细胞  
编辑。CSV 变成  
表而不是  
原始文本。PDF 渲染  
直接地。幻灯片可以  
被创建并且  
审查没有  
离开应用程序。

[操作网络表面]  
应用内浏览器  
制作小网  
更有用的工件：  
index.html，故事书，  
远程工作室，  
Slidev、Streamlit、  
和朱皮特。  
最小版本  
通常就足够了。  
单个index.html  
使用 JavaScript 文件  
CSS 可以变成  
一个活的表面  
用于互动  
并审查。

[审查变更]  
你和Codex  
可以看看  
同一对象同时  
这项工作  
仍在移动。  
评论变成  
指示。  
神器  
成为上下文。

侧面板是Codex的地方 不再只是一个聊天应用程序 开始成为这个地方工作发生了。

来源[刘杰生]

# 刘杰森的 练习表演 多快 Codex可以 [构建系统] 为了工作。

通过新的方式来移动项目 前进，任务变得更容易 拿起、回顾并继续 而不失去上下文。人们可以 花费更少的时间重新启动并且 花更多的时间在现有的基础上 已经在行动了。



OpenAI